

HelixCluster

HelixCluster

Размеркаваная аперацыйная сістэма для AI-вылічэнняў — ад датацэнтравых GPU да мабільных прылад на краі сеткі пад адзінай панэллю кіравання.

Зводка

Helix Cluster OS — гэта аперацыйная сістэма наступнага пакалення для размеркаваных вылічэнняў, якая аркеструе працоўныя нагрузкі на розна тыпных вузлах — ад датацэнтравых GPU да адзінаплатавых камп'ютараў на краі сеткі і мабільных прылад, аб'ядноўваючы HPC-дыспетчараванне, аркестрацыю кантэйнераў, AI/ML-інферэнцыю, федэратыўнае кіраванне некалькімі кластарамі і бяспечныя сесіі з некалькімі карыстальнікамі пад адзінай панэллю кіравання.

Кароткае апісанне

Размеркаваная АС на базе Go / кластар сумеснага выкарыстання GPU-рэсурсаў. Яна аб'ядноўвае HPC-дыспетчараванне (двухузроўневы дыспетчар у мадэлі Omega), аркестрацыю кантэйнераў, маршрутызацыю AI-інферэнцыі, федэрацыю і бяспечныя сесіі з некалькімі карыстальнікамі на розна тыпных вузлах, каардынаваныя праз SWIM-шаштанне і кансэнсус Raft з постквантавым шыфраваннем ад канца да канца.

Падрабязнае апісанне

Helix Cluster OS аркеструе працоўныя нагрузкі на радыкальна розна тыпным абсталяванні — ад датацэнтравых GPU да адзінаплатавых камп'ютараў на краі сеткі і нават мабільных прылад — пад адзінай панэллю кіравання, разглядаючы стойку з A100 і жменю SBC як адзіную даступную сетку

замест дванаццаці несумяшчальных астравоў. Гэта Го-працоўная прастора (монарэпа з git-падмадулямі), якая рэалізуе сяміўзроўневы стэк — ад узроўню L0 (апаратнае забеспячэнне) да ўзроўню L7 (федэрацыя і назіральнасць), каардынаваны чатырнаццацю мікрасэрвісамі панэлі кіравання. Удзельніцтва вузлоў адсочваецца праз SWIM-шаштанне і аўтаматычнае выяўленне, што дазваляе сетцы самааднаўляцца пры далучэнні ці адключэнні вузлоў; моцна ўзгоднены стан захоўваецца праз кансэнсус Raft, арганізаваны ў выглядзе Raft-груп па шардах з лакальнымі чытаннямі праз трымальнікаў лізінгу для хуткасці і STONITH-агароджай, каб гарантаваць, што падзелены вузел не зможа сапсаваць агульны стан. Размяшчэнне працоўных нагрузак ажыццяўляецца праз двухузроўневы дыспетчар у мадэлі Omega — аптымістычная канкурэнтнасць, адпаведнасць ClassAd, групавое дыспетчараванне, прыярытэтнае перанакіраванне з каэфіцыентам каштоўнасці і размяшчэнне на аснове абмежаванняў — і выходзіць за рамкі класічнага НРС-дыспетчара: маршрутызацыя з улікам вугляроднага следу і кошту/агульнай вартасці валодання, аўтаматычнае маштабаванне з пераходам у воблака і адаптары рынкаў (Akash, io.net, RunPod, AWS Spot, Chutes), якія дазваляюць задачы пераносіць на арэндаваныя рэсурсы, калі лакальныя сродкі вычарпаны.

Карыстальнікі не бачаць гэтай тэхнічнай машынерыі непасрэдна: яны ўзаемадзейнічаюць праз чыстую мадэль сесій (выдзяленне вылічальных рэсурсаў), інтэрактыўны WebSocket/PTY-тэрмінал, унутраны маршрут AI-інферэнцыі і чытанні выкарыстання пулаў. Бяспека з’яўляецца прыярытэтным узроўнем, а не дадаткам: ідэнтыфікацыя SPIFFE, пацвярджэнне прылады (выклік/адказ, доказ GPU-работы, замыканне), шлюз KYS для кантролю экспарту і постквантавы транспарт з шыфраваннем ад канца да канца на базе гібрыднага абмену ключамі X25519 + ML-KEM-768 з абаронай AEAD-запісаў і адхіленнем паўтораў — распрацаваны такім чынам, каб захоплены сёння трафік заставаўся канфідэнцыйным нават перад квантавым супернікамі будучыні. Карэктнасць не дэкларуецца, а *дэманструецца*: дэтэрмінісцкае тэставанне сімуляцыяй (запускі з фіксаванымі наборам зыходных дадзеных у стылі FoundationDB, ін’екцыя збояў, сімуляцыя сеткі, паўтор на ўзроўні байтаў і праверка лінеарызаванасці з дапамогай Ropcurve) узнаўляе размеркаваныя збоі на заказ, а абавязковае парнае мутацыйнае тэставанне пацвярджае, што тэсты-ахоўнікі сапраўды працуюць. Архітэктара і дакументацыя застаюцца адпаведнымі рэчаіснасці дзякуючы аўтаматычным праверкам, якія перарываюць зборку, калі рэальнасць і дакументацыя разыходзяцца.

Змест

Чаму мы яго стварылі

Каб запусаць AI і HPC-нагрузкі на кардынальна розных узроўнях абсталявання без неабходнасці злучаць асобныя дыспетчары, аркестратары і стэкі інферэнсу — і рабіць гэта з інжынернай гарантыяй, што кожная выпушчаная функцыя дэманструе *рэальную паводзіны канчатковага карыстальніка* (ніколі не зялёныя тэсты замест рэальных даных) і што кожная магчымасць, спецыфічная для аперацыйнай сістэмы, выкарыстоўвае сапраўдны натыўны механізм для кожнай платформы (без імітацый толькі для Linux). Галоўная праблема, сфармуляваная ў кіраванні рэпазіторыя, — гэта рэжым адмовы тыпу «тэсты прайшлі, але функцыя на самой справе не працуе», які праект створаны менавіта для таго, каб выключыць.

Чаму гэта змяняе правілы гульні

Ён аб'ядноўвае пяць звычайна асобных стэкаў — дыспетчараванне HPC, аркестрацыю кантэйнераў, інферэнс AI, федэрацыю некалькіх кластараў і бяспечныя шматкарыстальніцкія сесіі — у адзіную панэль кіравання, якая распасціраецца ад GPU ў датацэнтрах да мабільных прылад на краі сеткі. І робіць гэта з бюджэтам строгасці, які звычайна прымяняецца толькі да спецыялізаванай інфраструктуры: карэктнасць узроўню фармальных метадаў (спецыфікацыі TLA+, дэтэрмінісцкае мадэляванне, праверка лінеарызаванасці) і постквантавы канфідэнцыйны транспарт — тыя гарантыі, якія большасць аркестратараў нават не спрабуе забяспечыць. Апроч тэхнічных адрозненняў, размяшчэнне з улікам кошту і вугляроднага следу, а таксама магчымасць аўтаматычнага пераходу на рэсурсы воблачнага рынку робяць яго *эканамічным* рычагом: дыспетчар можа аўтаматычна шукаць таннейшыя, экалагічна чыстыя ці свабодныя магутнасці, таму адна і тая ж нагрузка каштуе менш і выкідвае менш CO₂ без неабходнасці перапісваць задачу.

Што новага

- **Дэтэрмінісцкае сімуляцыйнае тэсціраванне (DST)** — сімулятар з фіксаваным наборам выпадковых значэнняў, які цалкам паўтарае вынікі, уводзіць збоі, разыходжанне гадзіннікаў і падзелы сеткі, паўтарае іх байт за байтам і прапускае вынік праз правершчык лінеарызаванасці Rocurpine, каб адзін раз знойдзены хайзенбаг можна было паўтараць па запыце назаўжды.
- **Двухузроўневы дыспетчар паводле мадэлі Omega** — размяшчэнне з аптымістычнай канкурэнцыяй, падбор

паводле ClassAd, групавое дыспетчараванне і прыярытэтнае перанакіраванне з улікам каэфіцыента каштоўнасці, дызайн з агульным станам, які дазваляе многім дыспетчарам працаваць з адным кластарам без цэнтральнага вузкага месца.

- **Постквантавы E2EE / канфідэнцыйны інферэнс** — гібрыдны абмен ключамі X25519 + ML-KEM-768 з прывязкай ключа адказу да кожнага запыту і AEAD з абаронай ад паўтору (крыптапрымітывы рэальныя і правяраныя; поўны канфідэнцыйны абмен паміж некалькімі вузламі застаецца *запланаваным/абмежаваным*).
- **Давер на аснове пацвярджэнняў** — вузлы павінны *даказаць*, што яны сабой уяўляюць: ідэнтыфікацыя SPIFFE, доказ працы GPU, замыканне прылады, шлюз KYC для кантролю экспарту і генерацыя дакументаў для адпаведнасці Закону ЕС аб AI, таму давер засноўваецца на доказах, а не на здагадках аб становішчы ў сетцы.
- **Аркестрацыя з улікам кошту і вугляроднага следу** — мадэляванне агульнага кошту валодання, размяшчэнне з улікам вугляроднага следу, пераход на воблачныя рэсурсы, рэзерв N+K для адмоваўстойлівасці і адаптары для воблачных рынкаў, што робіць кошт і выкіды CO₂ першачарговымі ўваходнымі параметрамі дыспетчаравання замест другараднасці.
- **Мульти-Raft кансэнсус** — групы Raft для кожнага сегмента з лакальнымі чытаннямі праз трымальніка ліза, што забяспечвае нізкую затрымку кансістэнтнасці, падтрыманыя механізмам STONITH (IPMI / EC2 / Azure / SBD), каб завіслы вузел быў вырашана выдалены, а не пакінуты для парушэння стану.
- **Механічныя праверкі супраць дрэйфу** — archlint перапыняе зборку, як толькі задокументаваны кампанент адпавядае шляху пакета, якога не існуе, а рухавік дакументацыйнага ланцужка падтрымлівае байтавую кансістэнтнасць паміж Markdown / HTML / PDF / DOCX, каб дакументацыя не магла ціха падманваць код.

Змесціва

Найбольшыя тэхнічныя праблемы і спосабы іх вырашэння

- **«PASS-bluff» (тэсты праходзяць на нефункцыянальных магчымасцях)**. Гэта той рэжым адмовы, які ўвесь праект закліканы знішчыць: зялёны набор тэстаў на заглушках. Вырашана з дапамогай абавязковага парнага мутацыйнага тэсціравання — кожны элемент працы суправаджаецца тэстам-ахоўнікам з указаннем імя, які *павінен праваліцца* пры незалежнай мутацыі кода, перш чым элемент можа быць пазначаны як

завершаны. Такім чынам, праходны тэст даказвае, што правяраецца рэальная паводзіна, а не макет.

- **Пераанаснасць паміж платформамі (няма толькі-Linux-макетаў).** Вырашана з дапамогай агульнага інтэрфейсу, падзеленага паводле тэгаў зборкі на сапраўдныя сродкі для кожнай АС: Linux cgroup / /proc / ядро WireGuard, macOS sysctl / vm_stat / IOKit / wireguard-go. Затым правяраецца незалежным араклам АС, каб кожная платформа паведамляла сапраўдныя натыўны стан, а не выдумку Linux.
- **Распаўсюджаная карэктнасць пры збоях.** Вырашана з дапамогай дэтэрмінісцкага сімуляцыйнага тэсціравання і праверкі лінеарызаванасці, якія ствараюць і прайграюць раздзяленні, крахі і зрушэнні гадзіннікаў. Асновай з'яўляюцца фармальныя спецыфікацыі TLA+, якія фіксуюць інварыянты кансэнсусу і планавання яшчэ да напісання радка кода.
- **Адхіленне дакументацыі і архітэктуры.** Вырашана з дапамогай archlint, які перапыняе зборку пры выяўленні адлюстраванага, але неіснуючага пакета, і варот праверкі docs_chain без магчымасці абмінуць — адхіленне прыводзіць да памылкі зборкі, а не да састарэлай старонкі вікі.
- **Часнае апісанне незавершанай працы.** Канфідэнцыйны шматвузлавы кругазварот інферэнцыі наўмысна закрыты за білетам і пазначаны як «пакуль не правераны ад канца да канца», а не пададзены як гатовы. Тая ж дысцыпліна прымяняецца да таго, што *яшчэ не зроблена*, як і да таго, што зроблена.

Тэхналагічны стэк

- **Go (go.mod: 1.25 / toolchain 1.26.4)** — мова кіравальнай плоскасці ў працоўнай прасторы з ~30 модулямі. Абраная за танную канкурэнтнасць з дапамогай гаруцін і статычных бінарных файлаў, якія разгортваюцца аднолькава ад датацэнтра да краю сеткі.
- **Zig (0.14+) + C/C++** — выкарыстоўваецца там, дзе рантайм Go перашкаджае: нізкаўзроўневыя сістэмныя прымітывы і ядры GPU, якія патрабуюць дэтэрмінісцкага, свабоднага ад выдзяленняў кантролю над абсталяваннем.
- **gRPC + Protocol Buffers** — кожны міжсістэмны API (api/v1/) з'яўляецца тыпізаваным версіяваным кантрактам, што дазваляе чатырнаццаці мікрасэрвісам развівацца без парушэнняў і стварэння ўласных фарматаў перадачы даных.
- **Raft (etcd-raft) + SWIM gossip** — наўмыснае падзяленне: Raft захоўвае стан, які *павінен быць строга кансістэнтным*, у той час як SWIM gossip апрацоўвае членства і выяўленне ў маштабах, дзе кансэнсус быў бы занадта цяжкім.

- **PostgreSQL 16, Redis 7 cluster, etcd v3.5, SQLite** — правільнае сховішча для кожнай задачы: PostgreSQL для трывалага рэляцыйнага стану, Redis для гарачай кэш-памяці, etcd для каардынацыі, убудаваны SQLite для лакальнага рээстра працоўных элементаў НХС.
- **NATS 2.10 (JetStream), Kafka 4.0 (KRaft), RabbitMQ 3.13** — тры сістэмы абмену паведамленнямі для трох тыпаў трафіку: NATS/JetStream для хуткай унутранай падзеі, Kafka для трывалага высокапрадукцыйнага стрымінгу, RabbitMQ для класічнай семантыкі брокера.
- **WireGuard mesh + ML-KEM-768/X25519 + AES-256-GCM/ChaCha20-Poly1305 + HKDF** — WireGuard для лёгкай сеткі вузел-да-вузла, абгорнутай у гібрыдны постквантавы рукапацісканне і AEAD-запісы, каб транспарт быў абаронены ад класічных і квантавых атакавальнікаў.
- **SPIFFE + JWT (HS256) + RBAC на аснове скоўпаў + OPA** — шматслойная ідэнтыфікацыя і аўтарызацыя: SPIFFE для ідэнтыфікацыі нагрузкі, JWT для токенаў, RBAC на аснове скоўпаў для грубага доступу, OPA для выражэння дэталёвай палітыкі ў выглядзе кода.
- **Prometheus v2.50, Grafana 10.4, Jaeger 1.55, W3C tracing** — метрыкі, панэлі кіравання і размеркавання трэйсы з прапагацыяй кантэксту W3C, каб запыт можна было адсочваць праз сэрвісы і ўзроўні абсталявання.
- **HashiCorp Vault 1.16** — сакрэты і ключавы матэрыял захоўваюцца па-за кодам і канфігурацыяй і выдаюцца пад аўдытам.
- **Docker Compose, Kubernetes (kustomize, узмоцнены securityContext), Helm** — Compose для лакальнага запуску і Kubernetes/Helm з узмоцненымі кантэкстамі бяспекі для рэальных разгортванняў. Адна азначэнне прасоўваецца праз усе асяроддзі.
- **React + TypeScript + Vite (Node 20+)** — хуткі, тыпабяспечны вэб-інтэрфейс для сеансаў, тэрміналаў і выкарыстання пулаў.
- **TLA+** — фармальная спецыфікацыя інварыянтаў кансэнсусу і планавання, каб найбольш складаныя для тэсціравання ўласцівасці

Змест

Статус і заўвагі да сумленнасці

- **Статус: у распрацоўцы.** Гэта ранняя версія (0.1.0-dev). Некалькі пашыраных функцый — поўны канфідэнцыйны шматвузлавы інферэнсны цыкл, разлікі на рынку і фарміраванне раскладаў на аснове пацверджанняў — азначаны як ЗАПЛАНАВАНЫЯ / абмежаваныя інфраструктурай у рэпазіторыі і **не** падаюцца як цалкам працуючыя. Паказчыкі пакрыцця паведамляюцца аўтарамі.

- **Ліцензія: яшчэ не вызначана.** Не прапісана адназначна; URL-адрасы HelixCluster/HelixCluster і helixcluster.io у дыяграме Helm з’яўляюцца неправеранымі застаўкамі, якія не супадаюць з рэальнымі адаленымі рэсурсамі.
- Уключаныя праекты стэка LLM (LLMOrchestrator, LLMProvider, LLMsVerifier) — гэта аўтаномныя падмадулі, а не мадэльныя серверы, разгорнутыя ўнутры кластара.

Прыярытэтны ўзровень: Helix-асноўны (кластар інфраструктуры LLM — вылічальная аснова, здольная прымаць інфэрэнсныя і вылічальныя нагрузкі). Пасля HelixTrack.